ПРАКТИЧЕСКАЯ РАБОТА №5

Устойчивый многоклиентский TCP-сервер NetLab

Выполненные задания • исходный код • проверка • ответы на вопросы

Дисциплина: сетевое программирование на Node.js

Среда тестирования: Node.js v22.16.0

Автоматические проверки: 18 успешных

Устойчивый многоклиентский TCP-сервер NetLab

Цель: создать TCP-сервер, сохраняющий работоспособность при ошибках, отключениях и бездействии отдельных клиентов; обеспечить независимое состояние и ограничение входящих данных.

Среда: Node.js v22.16.0; встроенные модули `node:net` и `node:readline`; адрес `127.0.0.1`; TCP-порт `3020`.

1. Структура проекта и основные параметры

```
netlab-05/
  server.mjs           TCP-сервер
  client.mjs           интерактивный TCP-клиент
  tests/smoke.mjs      автоматические испытания
  отчёт/               отчёт PDF, Markdown, журнал тестов
  README.md            инструкция
```

Принятые значения: `MAX_CLIENTS = 5` (после самостоятельной доработки), `IDLE_MS = 20000` (20 секунд), `MAX_LINE_BYTES = 1024`, предупреждение примерно за 5 секунд до тайм-аута. Для быстрых тестов допускается временно задать `NETLAB_IDLE_MS=1400`; обычный запуск использует 20 секунд.

Сервер использует `Map<socket, state>`. Состояние содержит ID пользователя, буфер неполной строки, персональный счётчик команд, таймер предупреждения и признак закрытия. Сегменты TCP объединяются до `\n`, а длина проверяется **в байтах UTF-8** до преобразования буфера в текст. При закрытии сокет удаляется из `Map` и таймер отменяется.

2. Реализованные команды

Команда	Ответ / действие
`PING`	`PONG`
`STATS`	`STATS clients=N commands=M`, где M — число команд именно этого клиента
`SAY Привет`	`SENT` отправителю, `[user-N] Привет` другим клиентам
`WHO`	Список `user-N:порт` для активных клиентов
`HELP`	Перечень команд
`IDLE`	`IDLE 20 секунд` (обычный режим)
`QUIT`	`BYE` и корректное закрытие
`HELLO` и иная неизвестная	`ERROR UNKNOWN_COMMAND`; TCP-сеанс сохраняется

3. Опыты по устойчивости (задания 4 и 5)

Сценарий	Проверенный результат
Пять одновременных подключений	Все получили уникальные `WELCOME user-N`
Попытка шестого подключения	`ERROR SERVER_FULL`; первые пять остались подключены
Ошибка команды `HELLO`	`ERROR UNKNOWN_COMMAND`, затем `PING` → `PONG`
Превышение 1024 байт UTF-8	`ERROR LINE_TOO_LONG`; другой клиент продолжил работу
`QUIT` одного клиента	Получено `BYE`, оставшийся клиент ответил на `PING`
Бездействие	`WARNING IDLE` перед `ERROR IDLE_TIMEOUT`, на ускоренном тесте `NETLAB_IDLE_MS=1400`
Сервер отсутствует	Клиент получает `ECONNREFUSED`
Два сервера на одном порту	Второй выводит `EADDRINUSE`

Важно: проверка тайм-аута в журнале автоматизирована **с сокращённым значением 1,4 с**, чтобы не задерживать тест. Значение 20 секунд по умолчанию и его отображение командой `IDLE` проверены отдельно.

4. Самостоятельная часть и повышенная сложность

Выполнены все пять пунктов: `WHO`, `HELP`, лимит 5 клиентов, дата и время в логах `[CONNECT]`/`[CLOSE]`, команда `IDLE`. Повышенная сложность тоже реализована: предупреждение `WARNING IDLE` формируется через асинхронный `setTimeout`, без блокировки Event Loop. После предупреждения клиент может отправить команду и продлить время подключения.

Пример диалога:

```
Клиент 1: PING       -> PONG
Клиент 1: WHO        -> WHO user-1:49152, user-2:49153
Клиент 1: SAY Привет -> SENT
Клиент 2:              [user-1] Привет
Клиент 1: HELLO      -> ERROR UNKNOWN_COMMAND
Клиент 1: QUIT       -> BYE
```

Порты 49152 и 49153 приведены **как пример формата**: фактические исходящие порты выбирает ОС.

5. Ответы на вопросы для закрепления

1. Зачем хранить соединения в `Map`? Чтобы находить активные сокеты, быстро узнавать их количество, отправлять сообщения всем участникам и хранить персональное состояние.

2. Почему `pending` должен быть отдельным для каждого клиента? Потому что TCP разбивает данные на произвольные фрагменты; общая переменная перемешала бы строки разных соединений.

3. Чем `error` отличается от `close`? `error` сообщает о сетевой/сокетной ошибке; `close` уведомляет, что ресурс соединения уже закрыт. `close` возможен и без ошибки.

4. Закрывает ли `socket.setTimeout()` соединение автоматически? Нет; он генерирует событие `timeout`. Приложение должно само вызвать `socket.end()` или `socket.destroy()`.

5. Зачем ограничивать число клиентов и размер команд? Чтобы один пользователь или поток слишком больших данных не занимал чрезмерно память, сокеты и вычислительные ресурсы сервера.

6. Почему отключение одного клиента не должно завершать сервер? У каждого клиента свой сокет и обработчики; ошибка отдельного соединения не относится к слушающему сокету остальных пользователей.

7. Что будет, если забыть удалить закрытый сокет из коллекции? Возникнет утечка памяти и неправильный подсчёт активных подключений; новые клиенты могут отклоняться при фиктивно заполненном лимите.

8. Как Event Loop помогает обслуживать несколько соединений? Node.js реагирует на готовность событий ввода-вывода и выполняет их обработчики без отдельного блокирующего цикла на каждый сокет.

6. Проверки и вывод

Автоматический скрипт `node tests/smoke.mjs` на Node.js v22.16.0: **18 из 18 проверок успешно**. Журнал сохранён в `отчёт/результаты-тестов.txt`. Он подтверждает взаимодействие через настоящий loopback TCP, а не просто ожидаемые выводы.

Вывод: реализован устойчивый сервер с независимыми клиентскими сессиями, обработкой неизвестных команд и ошибок, ограничением подключений и размера данных, а также автоматическим завершением неактивных соединений. Программа продолжает обслуживать клиентов после сбоев отдельных сеансов.